NeuroGraph: A Temporal Knowledge Graph Operating System for AI Memory with Self-Evolving Retrieval

Ingest Anything, Remember Everything, Forget Intelligently, Reason Visually

Ashutosh Kumar Singh
ashutosh@neurograph.dev
NeuroGraph AI
https://github.com/neurographai/neurograph

March 2026

Abstract

We present **NeuroGraph**, a production-grade knowledge graph operating system implemented in Rust that unifies temporal knowledge management, multi-strategy hybrid retrieval, and biologically-inspired memory evolution within a single, ergonomic framework. NeuroGraph introduces several novel contributions: (1) a *richly-typed graph data model* with 16-field entities, bi-temporal relationships, episodic provenance, saga-based conversation threading, and a dual-mode ontology system (prescribed vs. learned types); (2) a *seven-strategy hybrid retrieval engine* combining semantic vector search, BM25 keyword indexing, Personalized PageRank, graph traversal, cross-encoder reranking, Maximal Marginal Relevance (MMR) for diversity, and node-distance graph-proximity reranking, all fused via Reciprocal Rank Fusion (RRF); (3) a *composable search configuration system* with per-channel (entity/relationship/episode/community) independent method and reranker selection, multi-channel typed results with provenance scoring, and 15+ pre-built search recipes; (4) a *two-level query engine* with a strategy-dispatch router (Local/Global/Temporal/MultiHop) and a MAGMA-style intent router classifying eight distinct query intents across four orthogonal graph views (semantic, temporal, causal, entity); (5) a *tiered memory hierarchy* (L1–L4: Working, Episodic, Semantic, Procedural) with automatic promotion, demotion, and Ebbinghaus-curve forgetting; (6) an *RL-guided memory evolution engine* using Q-table reinforcement learning for keep/forget decisions; (7) a *graph-level forgetting engine* with exponential importance decay, access-frequency boosting, and TTL-based expiration; (8) a *Git-like branch isolation protocol* with copy-on-write deltas, diffing, and merging; (9) a *provider-agnostic LLM abstraction* with a dyn-compatible trait, OpenAI-format universal client, SHA-256 keyed response caching with LRU eviction, and per-prompt-type token usage tracking; and (10) a *research paper intelligence platform* with PDF ingestion, multi-source academic search, RAG chat, and Model Context Protocol (MCP) integration for Claude, Cursor, and VS Code Copilot. The system is architected as a Rust workspace of four crates totaling over 35,000 lines of code, with zero-configuration defaults, pluggable storage backends (in-memory DashMap, embedded Sled, planned Neo4j), and a REST API with an embedded visualization dashboard. We describe the architecture, algorithms, and implementation in detail.

Keywords: Knowledge Graph, Temporal Reasoning, Graph RAG, Hybrid Retrieval, Memory Architecture, Reinforcement Learning, Multi-Agent Systems, Rust

1 Introduction

The proliferation of large language models (LLMs) has exposed a fundamental limitation: LLMs lack persistent, structured memory. While retrieval-augmented generation (RAG) systems partially address this by grounding responses in external documents, they treat knowledge as flat text chunks, discarding the relational structure that makes knowledge *actionable*.

Knowledge graphs (KGs) offer a solution by representing information as typed entities connected by labeled relationships. However, ex-

isting KG systems suffer from several deficiencies: (1) they lack temporal awareness—facts are treated as eternally valid; (2) they employ single-strategy retrieval that fails on diverse query types; (3) they have no mechanism for intelligent forgetting, leading to unbounded memory growth; and (4) they provide no isolation mechanism for concurrent multi-agent access.

NeuroGraph addresses all four deficiencies through an integrated operating-system metaphor: knowledge is *ingested* through a multi-stage pipeline, *stored* in a temporal graph with bi-temporal validity intervals, *retrieved* through a five-strategy hybrid engine, *evolved* through RL-guided forgetting, and *isolated* through Git-like branching.

1.1 Design Philosophy

NeuroGraph follows three core principles:

1. **Zero-Configuration Start**: A single line—`NeuroGraph::builder().build().await?`—creates a fully functional instance with in-memory storage, local hash embeddings, and regex-based entity extraction. No API keys, no external services.
2. **Progressive Enhancement**: Each subsystem (LLM extraction, tiered memory, multi-graph memory, evolution) is independently opt-in via the builder pattern.
3. **Rust-Native Performance**: The entire system is implemented in safe Rust with async/await concurrency via Tokio, lock-free data structures (DashMap, parking_lot), and zero-copy serialization.

1.2 Contributions

This paper makes the following contributions:

- A **complete system architecture** for a temporal knowledge graph OS with pluggable drivers, embedders, and LLM clients (§2).
- A **richly-typed graph data model** with bi-temporal entities, provenance episodes, dual-mode ontology, and schema registry (§2.5).
- A **pluggable driver trait** with 25 async methods and two built-in implementations: lock-free in-memory (DashMap) and embedded persistent (Sled B-tree) (§2.6).
- A **provider-agnostic LLM abstraction** with object-safe trait, OpenAI-format universal client, and per-call cost tracking (§2.7).
- A **two-level query engine** with strategy dispatch, token-budgeted context assembly, and per-query cost enforcement (§2.8).
- A **multi-stage ingestion pipeline** with extraction, ontology validation, embedding-based deduplication, and temporal conflict resolution (§3).
- A **seven-strategy hybrid retrieval engine** with RRF fusion, MMR diversity reranking, node-distance graph-proximity reranking, episode-mentions citation reranking, and DRIFT adaptive local/global search (§4).
- A **composable search configuration system** with per-channel method and reranker selection, multi-channel typed results, and 15+ pre-built search recipes (§4.7).
- A **biologically-inspired memory system** combining tiered hierarchy, Zettelkasten self-organization, and RL-guided evolution (§5).
- A **multi-graph memory architecture** with intent-aware routing across four orthogonal graph views (§6).
- A **temporal subsystem** with bi-temporal validity, point-in-time snapshots, Git-like branching, hybrid logical clock, fact version chains, and graph-level forgetting (§7).
- A **research paper intelligence platform** with PDF parsing, multi-source search, RAG chat, and MCP integration (§9, §10.3).

2 System Architecture

2.1 Workspace Organization

NeuroGraph is organized as a Rust Cargo workspace comprising four crates:

Table 1: NeuroGraph workspace crate organization.

Crate	Responsibility
`neurograph-core`	Core library: graph, ingestion, retrieval, memory, temporal, PDF, papers, chat, server, embedders, community
`neurograph-cli`	Command-line interface with 12 subcommands
`neurograph-mcp`	Model Context Protocol server (stdio/SSE)
`neurograph-eval`	Evaluation harness and benchmarks

The core crate exposes 17 top-level modules, each encapsulating a distinct subsystem. The workspace uses Rust 2021 edition with a minimum supported Rust version (MSRV) of 1.82.

2.2 Layered Architecture

The system follows a strict layered architecture:

1. **Public API Layer** (`NeuroGraph` struct): Exposes `add_text()`, `add_json()`, `query()`, `at()`, `entity_history()`, `what_changed()`, `remember()`, `recall()`, `detect_communities()`, and builder methods.
2. **Engine Layer**: Query routing (`QueryRouter`), ingestion pipeline (`IngestionPipeline`), community detection.
3. **Service Layer**: LLM clients (OpenAI, Ollama), embedding providers (OpenAI, Ollama, FastEmbed/Hash, cached), paper search APIs.
4. **Driver Layer**: Pluggable storage backends via the `GraphDriver` trait—`MemoryDriver` (in-memory with `DashMap`), `EmbeddedDriver` (Sled persistent B-tree), and planned Neo4j driver.

2.3 Builder Pattern and Configuration

The `NeuroGraphBuilder` provides a fluent API for progressive configuration:

```
let ng = NeuroGraph::builder()
    .name("research-kg")
    .embedded("./data")
    .budget(5.0)              // USD limit
    .multigraph()             // Enable MAGMA
    .tiered_memory()          // Enable L1-L4
    .evolution()              // Enable RL
        forgetting
    .build().await?;
```

The `.full()` shorthand enables all v0.2 subsystems simultaneously. When no configuration is provided, `NeuroGraphConfig::zero_config()` creates sensible defaults: in-memory storage, local hash embeddings (128-dimensional), no LLM (regex extraction), and a $10 USD budget ceiling.

2.4 Error Model and Cost Tracking

NeuroGraph employs a typed error hierarchy via `thiserror`:

- `NeuroGraphError::Driver` — storage failures
- `NeuroGraphError::Llm` — LLM API errors
- `NeuroGraphError::Embedder` — embedding failures
- `NeuroGraphError::BudgetExceeded{spent, limit}` — hard budget enforcement
- `NeuroGraphError::Temporal` — date parsing, snapshot
- `NeuroGraphError::Community` — detection/-summarization

Every LLM call records its cost via the `CostTracker`, and the `ConcurrencyLimiter` throttles parallel LLM calls to prevent rate-limit breaches.

2.5 Graph Data Model

The `graph` module defines the core data structures that underpin all subsystems. Each type uses UUID v4 identifiers, `serde` serialization, and the builder pattern.

Entity. The `Entity` struct carries 16 fields: `id` (UUID), `name`, `entity_type` (wrapped `EntityType(String)` for type safety), `summary` (LLM-generated regional description), `name_embedding` (`Option<Vec<f32>>`), `group_id` (multi-tenant partition), `labels` (flexible categorization, e.g., `["Entity", "Person"]`), `attributes` (`HashMap<String, serde_json::Value>`), `importance_score` ($[0, 1]$, PageRank + access frequency), `access_count`, `community_ids` (`HashMap<u32, String>` mapping hierarchy level to community), `created_at`, `updated_at`, and `metadata`. The `record_access()` method atomically increments the access counter and touches the modification timestamp.

Relationship. Edges carry bi-temporal fields directly from Graphiti's `EntityEdge`: `valid_from` / `valid_until` (real-world validity), `created_at` / `expired_at` (system time). Additional fields include `fact` (natural language statement), `fact_embedding`, `weight` (default 1.0), `confidence` ($[0, 1]$), and `episode_ids` (provenance chain). The `is_valid_at(t)` method enables point-in-time queries, and `invalidate()` sets both `valid_until` and `expired_at`.

Episode. Episodes are immutable provenance records with an `EpisodeType` enum supporting six source types: `Message`, `Json`, `Text`, `Image`, `Audio`, and `Pdf`. Each episode carries `valid_at` (real-world timestamp of the source data) and `entity_edge_ids` (extracted relationship UUIDs).

Saga (Conversation Thread). The `Saga` struct groups `Episodes` into ordered conversation threads, closing a critical competitive gap with Graphiti's `SagaNode`. Each saga maintains: `SagaId` (UUID), `name`, optional `description` and `source` identifiers, `group_id` for multi-tenant isolation, and an append-only `episode_ids` vector that preserves chronological insertion order.

The graph structure uses two well-known edge types: `HAS_EPISODE` (with a `position` attribute) from Saga to Episode, and `NEXT_EPISODE` between consecutive episodes. The `episode_pairs()` method generates $(\text{source}, \text{target})$ pairs for constructing the linked-list threading edges. This enables queries such as "What was discussed in conversation X?" and chronological replay of multi-turn interactions.

TemporalFact. The `TemporalFact` struct wraps a relationship with a dedicated `TemporalValidity` struct (`valid_from`, `valid_until`, `recorded_at`, `invalidated_at`), plus `superseded_by` (`Option<RelationshipId>`) and `invalidation_reason`. This enables fact version chains where superseded facts link to their replacements.

Ontology. The `Ontology` system supports two modes: *prescribed* (user-defined `EntityTypeDefinition` with expected attributes, data types, required flags, and few-shot examples) and *learned* (auto-discovered types from extraction). `RelationshipTypeDefinition` specifies allowed source/target entity types. The `to_extraction_prompt()` method generates LLM-ready markdown from the ontology, and `strict_mode` constrains extraction to prescribed types only.

GraphSchema. The `GraphSchema` registry tracks all types currently present in the graph: entity type counts, relationship type counts, known type sets, and totals. It auto-registers learned types into the ontology via `record_entity_type()` and `record_relationship_type()`.

2.6 GraphDriver Trait and Implementations

The `GraphDriver` trait defines 25 `async` methods across six operation categories:

Table 2: GraphDriver trait method categories.

Category	Methods	Operations
Entity	5	store, get, batch get, delete, list
Relationship	4	store, batch store, get, get by entity
Episode	3	store, get, list recent
Community	5	store, get, by level, dirty, list
Search	3	vector entities, vector rels, text search
Graph/Temporal	3	traverse, snapshot_at, stats

Two implementations are provided:

- **MemoryDriver**: Uses `DashMap<K, V>` for lock-free concurrent access to entities, relationships, episodes, and communities. Vector search uses brute-force cosine similarity ($O(nk)$); text search uses case-insensitive substring matching. Temporal filtering applies `is_valid_at()` to all relationships in `snapshot_at()`.
- **EmbeddedDriver**: Uses Sled embedded B-tree with separate trees per data type (`entities`, `relationships`, `episodes`, `communities`). Serialization via `bincode` for

compact binary storage. Inherits the same vector/text search algorithms as MemoryDriver but reads from persistent disk.

`GraphDriverConfig` includes `storage_path`, `max_connections`, and optional `uri`/`username`/`password` fields for planned remote drivers (Neo4j, FalkorDB).

2.7 LLM Abstraction Layer

The `llm` module provides a provider-agnostic LLM interface with response caching and per-prompt-type token tracking.

LlmClient Trait. The core trait defines three methods: `model_name()`, `cost_per_token()` (returns `(input_cost, output_cost)` in USD), and `complete(request) -> CompletionResponse`. The trait is `dyn`-compatible (object-safe); structured JSON deserialization is provided via a free function `complete_structured<T>()` rather than a generic trait method, preserving dynamic dispatch.

CompletionRequest. Encapsulates `system_prompt`, `user_prompt`, `temperature`, `max_tokens`, and `json_mode` (requests `response_format: json_object`).

GenericLlmClient. Uses raw `reqwest` HTTP calls to any OpenAI-compatible endpoint (Ollama at `localhost:11434/v1`, vLLM, LiteLLM, Azure OpenAI). Features include: configurable timeout, bearer token auth, rate-limit detection (HTTP 429 → `LlmError::RateLimited`), and per-call cost calculation using `LlmConfig::calculate_cost(input_tokens, output_tokens)`.

LlmConfig Presets. Pre-built configurations: `gpt4o_mini()` ($0.15/$0.60 per million tokens), `ollama(model)` (zero-cost, local endpoint). Each config stores `input_cost_per_million` and `output_cost_per_million` for budget tracking.

Response Cache (`LlmCache`). Prevents redundant API calls for identical extraction prompts—closing a gap where GraphRAG provides a modular caching factory. Cache keys are computed deterministically via SHA-256 over the concatenation of prompt and model name. The cache supports:

- **LRU Eviction**: When the cache exceeds `max_entries` (configurable, default 1,000), the oldest entry by `cached_at` timestamp is evicted.
- **TTL Expiration**: Each `CachedResponse` carries an optional TTL (`chrono::Duration`). Expired entries are treated as cache misses and evicted on the next write pass.
- **Observability**: `CacheStats` tracks hits, misses, and evictions, with a `hit_rate()` convenience method. Thread-safety is provided via `tokio::sync::RwLock`.

Each cached entry records the response content, model name, input/output token counts, and timestamp—enabling cost tracking even for cached results.

Token Usage Tracker (`TokenTracker`). Tracks LLM token consumption at granular, per-prompt-type level. Eight built-in `PromptType` variants categorize API calls: `EntityExtraction`, `RelationshipExtraction`, `EntityResolution`, `CommunitySummary`, `QueryRewrite`, `AnswerGeneration`, `Reranking`, and `ConflictResolution`, plus `Custom(String)` for extension. Each `record()` call updates both per-type counters (`TokenUsage`: input/output tokens, call count, estimated cost) and global atomic counters (`AtomicU64` for lock-free reads). The `print_summary()` method renders a formatted table for cost analysis, and `get_summary()` returns the full `HashMap<PromptType, TokenUsage>` for programmatic access.

2.8 Query Engine

The `engine` module orchestrates query execution through a two-level routing architecture:

Level 1: QueryRouter (Strategy Dispatch). Classifies queries into four types via keyword heuristic scoring: `Local` (entity-focused, default), `Global` (dataset-wide: "summarize", "overview", "all"), `Temporal` ("when", "before", "history", "in 20XX"), and `MultiHop` ("connected", "related", "path"). This router dispatches to the appropriate `QueryStrategy` implementation:

- **LocalStrategy**: Hybrid retrieval → relationship fetch → context assembly → LLM answer generation. Returns raw context if no LLM is configured.

- **GlobalStrategy**: Community map-reduce pipeline— (1) retrieve community summaries, (2) map: LLM partial answer per community, (3) reduce: synthesize final answer. Falls back to hybrid search when no communities exist.

Level 2: IntentRouter (Graph View Selection). When multi-graph memory is enabled, the `IntentRouter` (§6) performs finer-grained 8-type intent classification and selects which graph views to query. The two routers are complementary: the engine router selects the execution strategy, while the intent router selects the data sources.

ContextBuilder. Assembles LLM prompts from graph data within a configurable token budget (default 4,000 tokens, estimated at ≈4 chars/token). Content is added in priority order: (1) entity descriptions with types and summaries, (2) relationship facts with temporal validity markers (✓ `current` vs. × `historical`), (3) community summaries for global context. Each section is truncated at the budget boundary.

QueryBudget. Per-query cost enforcement using atomic microdollar tracking (`AtomicU64` storing hundredths of cents). Provides `record_cost()`, `can_afford()`, and `reset()` operations.

2.9 Utility Infrastructure

ConcurrencyLimiter. Semaphore-based rate limiter (from Graphiti's `semaphore_gather()` pattern) wrapping `tokio::sync::Semaphore` with an atomic active-count tracker. Default limit: 10 concurrent LLM calls.

CostTracker. Process-wide atomic cost accumulator storing microdollars (`Arc<AtomicU64>`). Provides `record()`, `total_cost_usd()`, `is_over_budget()`, and `remaining_usd()` for budget enforcement across all operations.

Hashing. Two-phase deduplication hashing from Graphiti: (1) deterministic SHA-256 content hash with text normalization (lowercase, whitespace collapse); (2) BLAKE3 fast hash for high-throughput scenarios. Helper functions `entity_dedup_key(name, type)` and `relationship_dedup_key(source, target, type)` generate consistent dedup keys.

Text Utilities. `normalize_name()` (lowercase, replace underscores/hyphens), `extract_terms()` (stopword-filtered tokenization), `truncate_to_tokens()` (word-boundary-aware truncation), `estimate_tokens()` ($\lceil$chars/4$\rceil$), and `chunk_text()` (sentence-boundary-respecting chunking).

3 Ingestion Pipeline

3.1 Pipeline Architecture

The ingestion pipeline transforms raw input (text, JSON, PDF) into a structured knowledge graph through six stages:

$$\text{Src} \xrightarrow{1} \text{Ep} \xrightarrow{2} \text{Ext} \xrightarrow{3} \text{Val} \xrightarrow{4} \text{Dup} \xrightarrow{5} \text{Cfl} \xrightarrow{6} \text{Str} \tag{1}$$

Stage 1: Episode Creation. Every ingestion creates an `Episode`—a provenance record linking the raw input to the entities and relationships it produces. Episodes carry a `group_id` for multi-tenant isolation and a `created_at` timestamp for temporal ordering.

Stage 2: Entity/Relationship Extraction. The `TextExtractor` supports two modes: (a) LLM-based extraction using structured prompts to an OpenAI-compatible API, returning typed entities with summaries and relationships with confidence scores; (b) regex-only fallback that identifies capitalized proper nouns and verb-mediated relationships, requiring no API keys.

Stage 3: Ontology Validation. The `SchemaValidator` checks extracted entities and relationships against a configurable `Ontology` that defines allowed entity types, relationship types, and constraints. Invalid items are logged and skipped; warnings are emitted for items requiring normalization.

Stage 4: Embedding-Based Deduplication. The `Deduplicator` prevents entity proliferation by computing cosine similarity between a new entity's name embedding and all existing entities of the same type. If similarity exceeds a threshold (default 0.85), the entities are merged:

$$\text{sim}(\mathbf{e}_{\text{new}}, \mathbf{e}_{\text{existing}}) > \tau \implies \text{Merge}(\mathbf{e}_{\text{existing}}, \mathbf{e}_{\text{new}}) \tag{2}$$

Merge combines summaries and preserves the existing entity's ID, ensuring referential integrity.

Stage 5: Temporal Conflict Resolution. The `ConflictResolver` detects four conflict types between a new relationship and existing ones sharing the same source entity and relationship type:

Table 3: Conflict types and resolution strategies.

Type	Resolution
Redundant	Skip (identical fact already exists)
Contradiction	Invalidate old by setting `valid_until = now`
Supersession	Same as contradiction (newer fact wins)
SoftConflict	Coexist (non-exclusive relationship)

Stage 6: Embedding and Storage. Each surviving entity receives a name embedding and each relationship a fact embedding. Both are stored via the `GraphDriver` trait.

3.2 JSON Ingestion

The `JsonExtractor` recursively walks JSON objects, creating entities from keys and relationships from nesting structure. The extracted graph is then routed through the same dedup/conflict/store pipeline as text extraction.

4 Hybrid Retrieval Engine

4.1 Seven-Strategy Pipeline

NeuroGraph's retrieval engine combines seven complementary strategies in a single pipeline, each addressing different aspects of relevance:

1. **Semantic Search**: Embeds the query via the configured embedder, then performs cosine similarity search over entity name embeddings using the driver's vector index. Default weight: 0.40.
2. **BM25 Keyword Search**: A full inverted index with TF-IDF scoring, supporting configurable k_1 (1.2) and b (0.75) parameters. Default weight: 0.25.
3. **Personalized PageRank (PPR)**: Seeded from the top-5 semantic results, PPR builds a local adjacency subgraph from 1-hop neighborhoods and runs power iteration (damping $\alpha = 0.85$, max iterations 20, convergence $\epsilon = 10^{-8}$). Default weight: 0.20.
4. **Graph Traversal**: BFS from user-specified seed entities up to 2 hops, scoring by hop distance with exponential decay. Default weight: 0.15.
5. **Cross-Encoder Reranking**: Either rule-based (zero-cost, offline) or API-based, reranking the fused candidate list for precision. Rule-based scoring combines query-document overlap, entity type boosting, summary length normalization, and original score blending.
6. **MMR Reranking**: Maximal Marginal Relevance (§4.3) for diversity-aware result selection.
7. **Node Distance Reranking**: Graph proximity scoring (§4.4) via BFS-based shortest-path distances.

4.2 Reciprocal Rank Fusion

Results from the retrieval stages are merged using Reciprocal Rank Fusion (RRF):

$$\mathrm{RRF}(d) = \sum_{s \in \mathcal{S}} \frac{w_s}{k + r_s(d) + 1} \tag{3}$$

where $\mathcal{S}$ is the set of strategies, w_s is the strategy weight (summing to 1.0), k is the RRF constant (default 60), and $r_s(d)$ is the rank of document d in strategy s's result list. Entities appearing in multiple lists receive additive score boosts, naturally promoting cross-strategy consensus.

4.3 Maximal Marginal Relevance (MMR)

MMR reranking balances relevance to the query with diversity among selected results—closing a critical competitive gap, as both Graphiti and GraphRAG support MMR-based reranking. The `MmrReranker` greedily selects items using:

$$\mathrm{MMR}(d) = \lambda \cdot \mathrm{sim}(d, q) - (1 - \lambda) \cdot \max_{d_j \in S} \mathrm{sim}(d, d_j) \tag{4}$$

where $\lambda \in [0, 1]$ controls the relevance-diversity trade-off: $\lambda = 1.0$ yields pure relevance (identical to top-k by score), $\lambda = 0.0$ maximizes diversity, and $\lambda = 0.7$ is the recommended default. The similarity metric is configurable between cosine similarity (default) and raw dot product.

The algorithm precomputes query-candidate similarities, then performs k greedy selection steps, each computing the maximum similarity between each unselected candidate and all already-selected items. Complexity is $O(k \cdot n \cdot d)$ where n is the candidate count and d is the embedding dimension.

4.4 Node Distance Reranker

The `NodeDistanceReranker` reranks results by graph distance from a center (focal) node, closing a gap where Graphiti has `NodeReranker.node_distance` and `EdgeReranker.node_distance`. Scores follow exponential decay:

$$\text{score}(v) = \gamma^{d(v,c)} \tag{5}$$

where γ is the decay factor (default 0.5) and $d(v, c)$ is the shortest BFS distance from node v to center c, capped at `max_depth` (default 3). A `combine_scores()` method blends distance scores with existing relevance scores: $\alpha \cdot s_{\text{relevance}} + (1 - \alpha) \cdot s_{\text{distance}}$.

4.5 Episode Mentions Reranker

The `EpisodeMentionsReranker` reranks results by how many episodes (provenance sources) reference them—a proxy for importance by citation frequency. Items mentioned across many episodes are likely more significant. Scores are normalized: the most-mentioned item receives 1.0, with others proportional. A `combine_scores()` variant blends mention counts with relevance via weight α. Any type implementing the `HasEpisodeIds` trait (providing `episode_ids()` and `episode_count()`) can be reranked.

4.6 DRIFT: Dynamic Reasoning and Inference with Flexible Traversal

DRIFT adaptively switches between local and global search strategies based on query characteristics. Query breadth is estimated via a composite heuristic:

$$\beta(q) = 0.7 \cdot \frac{|\text{broad_kw}(q)|}{|\text{broad_kw}(q)| + |\text{specific_kw}(q)|} + 0.3 \cdot \frac{|q|_{\text{words}}}{15} \tag{6}$$

- $\beta > \tau$: **Global** — rank community summaries by keyword overlap, collect entities from top-k communities, generate follow-up queries from key entities, refine locally.
- $\beta < 1 - \tau$: **Local** — BFS from query-matching seed entities with per-hop score decay of 0.7.
- Otherwise: **Adaptive** — execute both strategies, merge with dynamic weighting (local 0.6, global 0.4 when local finds results).

4.7 Composable Search Configuration

The `SearchConfig` system enables per-channel independent configuration of search methods and rerankers, modeled after Graphiti's composable `SearchConfig` but made Rust-idiomatic with the builder pattern.

Per-Channel Configuration. Each of the four channels—entities, relationships, episodes, communities—has an independent `ChannelConfig` specifying:

- `enabled`: whether the channel participates
- `methods`: one or more `SearchMethod`s (Cosine, Bm25, Bfs, Ppr, DriftAdaptive)
- `reranker`: the `RerankerType` (Rrf, Mmr{λ}, CrossEncoder{model}, NodeDistance{center_id}, EpisodeMentions, None)
- `limit`: per-channel result cap
- `sim_min_score`: minimum similarity threshold

Builder Pattern. The `SearchConfigBuilder` allows selective overrides:

```
let config = SearchConfig::builder()
    .entities(ChannelConfig {
        methods: vec![Cosine, Bm25],
        reranker: RerankerType::Mmr { lambda:
            0.7 },
        limit: 20,
        ..Default::default()
    })
    .communities(ChannelConfig::disabled())
    .build();
```

Pre-Built Recipes. The `SearchRecipes` module provides 15+ pre-configured `SearchConfig`s organized by use case (Table 4).

Table 4: Pre-built search configuration recipes.

Recipe	Description
`combined_hybrid_rrf`	All channels, RRF
`combined_hybrid_mmr`	All channels, MMR diversity
`combined_hybrid_cross_enc`	All channels, LLM reranking
`entity_hybrid_rrf`	Entity-only, RRF
`entity_hybrid_mmr`	Entity-only, MMR
`entity_hybrid_ep_mentions`	Entity-only, citation rerank
`entity_only_cosine`	Cosine only, no reranking
`rel_hybrid_rrf`	Relationship-only, RRF
`rel_hybrid_node_distance`	Rel-only, graph proximity
`graph_aware_ppr`	PPR+BFS (unique)
`drift_adaptive`	DRIFT+MMR (unique)
`neighborhood_search`	BFS, distance reranking
`conversation_replay`	Episode, BM25+cosine
`community_hybrid_rrf`	Community-only, RRF
`community_hybrid_mmr`	Community-only, MMR

4.8 Multi-Channel Search Results

The `SearchResults` struct provides typed, multi-channel result output—closing a gap where Graphiti returns separate lists for edges, nodes, episodes, and communities while NeuroGraph previously returned a flat entity list.

Each channel carries a `Vec<ScoredItem<T>` where `ScoredItem` attaches the original search score, source method name, and optional reranker score. The `effective_score()` method returns the reranker score when available, falling back to the original. `SearchMetadata` records the original query, config name, candidate count, search duration, and channels searched.

For backward compatibility, `SearchResults::flatten()` interleaves all channels into a single `Vec<FlatResult>` sorted by effective score descending, where each `FlatResult` carries an id, name, `Channel` enum variant, score, and snippet.

5 Memory Architecture

NeuroGraph implements a three-layer memory architecture inspired by cognitive science: tiered storage, self-organizing Zettelkasten, and RL-guided evolution.

5.1 Tiered Memory (L1–L4)

The tiered memory system maintains four levels, each with distinct characteristics:

Table 5: Tiered memory hierarchy.

Tier	Name	Cap	Role
L1	Working	100	Current context window, hot cache
L2	Episodic	10K	Recent interactions, time-indexed
L3	Semantic	∞	Knowledge graph facts
L4	Procedural	∞	Learned rules, heuristics

Promotion Policy. Items are promoted based on access frequency and importance score:

- L1 $\rightarrow$ L2: after 3 accesses
- L2 $\rightarrow$ L3: after 10 accesses *and* importance $\geq$ 0.6
- L3 $\rightarrow$ L4: after 50 accesses (confirmed pattern)

Demotion and Eviction. Items not accessed within a configurable window (default 168 hours / 1 week) with importance < 0.3 are evicted from L2. L1 uses FIFO eviction when at capacity, with evictees demoted to L2.

Search Tier Boosting. When searching across tiers, matches in higher tiers receive multiplicative boosts: L1 $\times$4.0, L2 $\times$2.0, L3 $\times$1.5, L4 $\times$1.0, ensuring that recently active memories surface first.

Procedural Rules. L4 stores `LearnedRule` objects with a pattern string, confidence score, usage counter, and source memory IDs. During query context assembly, applicable rules are retrieved via keyword matching and included in the LLM prompt.

5.2 Zettelkasten Self-Organizing Memory

Inspired by the A-MEM framework, the Zettelkasten module implements a self-organizing note graph where each `MemoryNote` carries:

- Auto-extracted keywords (stop-word filtered, max 20)
- Embedding vector for semantic similarity
- Bidirectional links to related notes
- Memory strength S (Ebbinghaus)
- Access count and temporal metadata

The A-MEM Insight: Keyword Evolution. When a new note n_{new} is added, the system: (1) finds semantically similar notes by cosine similarity above threshold τ_{link} (default 0.70); (2) creates bidirectional links; (3) *evolves existing notes' keywords* by injecting up to 3 novel keywords from n_{new}. This causes the knowledge graph to self-organize—older notes become richer as related knowledge accumulates.

Ebbinghaus Forgetting Curve. Each note's retention probability follows:

$$R = e^{-t/(S \cdot 24)} \tag{7}$$

where t is hours since last access and S is memory strength (starts at 1.0, increases by 0.5 per access, capped at 10.0). Notes with $R < \tau_{\text{forget}}$ (default 0.2) are candidates for removal. The forgetting pass also cleans up indexes and removes dangling links.

5.3 RL-Guided Memory Evolution

The `MemoryEvolution` engine runs periodic cycles with five steps:

1. **Composite Scoring**: Each item receives an importance score combining four signals:

   $$I = 0.3 \cdot f_{\text{freq}} + 0.3 \cdot f_{\text{recency}} + 0.25 \cdot f_{\text{conn}} + 0.15 \cdot f_{\text{salience}} \tag{8}$$

   where $f_{\text{freq}} = \ln(\text{accesses} + 1)/10$, $f_{\text{recency}} = 1/(1 + h/168)$ (1-week half-life), $f_{\text{conn}} = \min(\text{connectivity}, 20)/20$, and $f_{\text{salience}} = \min(\text{words}, 50)/50$.

2. **Temporal Decay**: Importance is multiplied by a decay factor D according to the configured function: exponential ($e^{-\lambda t}$), linear ($1 - \lambda t$), or step (0 if $t > \tau$).

3. **RL Keep/Forget**: Items below the forget threshold (default 0.1) are evaluated by the Q-table RL agent. State is discretized into bins of (importance, access frequency, recency, connectivity). The agent uses ϵ-greedy exploration ($\epsilon = 0.1$) with Q-learning updates:

   $$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{9}$$

   with $\alpha = 0.1$, $\gamma = 0.95$.

4. **Consolidation**: Items with cosine similarity > 0.95 are merged, keeping the higher-importance item.

5. **Overflow Eviction**: If items exceed `max_items` (default 100K), the lowest-importance items are truncated.

6 Multi-Graph Memory (MAGMA)

6.1 Four Orthogonal Graph Views

Inspired by the MAGMA architecture, each `MemoryItem` is indexed across four independent graph views:

1. **Semantic Graph**: Edges represent embedding cosine similarity above a threshold (default 0.75). Uses brute-force $O(n^2)$ comparison with planned HNSW upgrade.

2. **Temporal Graph**: Edges encode Allen's interval algebra relations (before, after, during, overlaps, etc.) between items' validity intervals.

3. **Causal Graph**: Directed edges represent cause-effect relationships extracted via the `CausalExtractor`, which applies five regex patterns with typed confidence scores: *caused/led to* (0.8), *because/since* (0.7), *therefore/consequently* (0.75), *if-then* (0.6), and *enabled/allowed* (0.7). Edges below the configurable threshold (default 0.6) are discarded. The graph supports both forward chaining (`traverse()`: BFS with per-hop confidence decay) and backward chaining (`root_causes()`: enumerates all root-cause paths up to a configurable depth).

4. **Entity Graph**: Standard typed entity relationships extracted from content.

Ingestion is parallelized via `tokio::join!`, concurrently indexing each item across all four graphs.

6.2 Intent-Aware Query Router

The `IntentRouter` classifies queries into eight intent types using keyword-scoring heuristics:

Table 6: Intent types and primary graph views.

Intent	Example	View
Factual	Who is Alice?	Entity
Temporal	What happened in 2024?	Temporal
Causal	Why did X fail?	Causal
Hypothetical	What if X hadn't...?	Causal
Comparative	Compare X and Y	Entity
Aggregative	How many at X?	Entity
MultiHop	Alice's boss reports to?	Entity
Exploratory	Tell me about X	Semantic

Each intent maps to a `TraversalPlan` specifying view weights, max hops (2–5), max nodes per graph (20–40), and fusion strategy (RRF with $k = 60$ or weighted sum).

6.3 Fusion Engine

The `FusionEngine` merges `SubgraphResults` from multiple views using type-aligned RRF with cross-view reinforcement. Items appearing in multiple views receive additive score boosts proportional to the view weights specified by the intent router.

7 Temporal Subsystem

7.1 Bi-Temporal Model

Every `Relationship` in NeuroGraph carries two temporal intervals:

- `valid_from` / `valid_until`: The real-world validity period of the fact.
- `created_at`: When the fact was ingested (system time).

The `TemporalManager` provides three temporal operations:

- `snapshot_at(t)`: Returns all entities and relationships valid at timestamp t, filtering by `valid_from` $\leq t$ and `valid_until` is null or $> t$.
- `what_changed(t1, t2)`: Returns entities added, modified, and relationships invalidated between t_1 and t_2.
- `build_timeline()`: Constructs a chronological event list for visualization (G6 Timebar integration).

7.2 Git-Like Branch Isolation

The `BranchManager` implements a copy-on-write (COW) branching model for multi-agent memory isolation:

- **Create**: Lightweight branch forked from a parent, storing only a delta set of added/removed entity and relationship IDs. Thread-safe via `DashMap`.
- **Read**: Visibility check walks the branch chain (branch delta → parent delta → root), implementing COW semantics.
- **Diff**: Computes the symmetric difference between two branches' delta sets.
- **Merge**: Applies source branch's deltas to target, detecting conflicts (e.g., entity added in source but removed in target). Conflicts are flagged but the merge proceeds.
- **Abandon**: Marks a branch as closed without merging.

Configuration enforces a maximum of 100 open branches and optional auto-abandonment after 24 hours.

7.3 Logical Clock and Versioning

The `LogicalClock` implements a hybrid logical clock combining wall time (`DateTime<Utc>`) with a process-global monotonic counter (`AtomicU64`). The `happened_before()` method compares wall times first and breaks ties with the counter, ensuring deterministic ordering even under concurrent ingestion.

The versioning module provides `FactVersionChain`—a chronological list of `FactVersion` records for a single relationship. Each version carries the fact text, validity interval, confidence, and an optional `superseded_by` link. The `version_at(t)` method returns the fact that was valid at any historical timestamp.

`EntityHistory` tracks modification records per entity, supporting six change types: `Created`, `SummaryUpdated`, `Merged` (deduplication), `TypeChanged`, `AttributesUpdated`, and `DecayMarked`. The `changes_between(t1, t2)` method enables audit trail queries.

7.4 Graph-Level Forgetting Engine

The `temporal::forgetting` module provides a separate, graph-level forgetting mechanism distinct from the memory-layer evolution engine

(§5). The `ForgettingEngine` operates directly on `Entity` nodes in the `GraphDriver`:

$$I_{\text{decay}}(e) = I_0(e) \cdot e^{-\lambda d} + \frac{\ln(a+1)}{10} + \frac{\ln(r+1)}{10} \tag{10}$$

where I_0 is the entity's base importance, λ is the `daily_decay_rate` (default 0.01), d is days since last access, a is the access count (log-scaled boost), and r is the relationship count (connectivity boost). Entities with $I_{\text{decay}} < \tau_{\text{prune}}$ (default 0.1) and access count < `min_access_count` (default 5) become prune candidates.

The `decay_pass(auto_prune, group_id)` method evaluates all entities, updates importance scores, and optionally deletes candidates in batches (`max_prune_batch`, default 100). A separate TTL-based expiration via `is_expired()` removes entities exceeding `default_ttl` regardless of importance.

8 Community Detection

NeuroGraph implements two community detection algorithms:

- **Louvain**: Greedy modularity optimization with configurable resolution parameter, max iterations, and minimum community size.
- **Leiden**: Refinement phase on top of Louvain for higher-quality, well-connected communities.

Both operate on the entity-relationship graph extracted from the `GraphDriver`. Detected communities are stored as `Community` objects with member entity IDs, hierarchical level, and optional LLM-generated summaries.

The `IncrementalCommunityUpdater` supports post-ingestion updates: given a set of newly added entity IDs, it re-evaluates only the affected neighborhoods rather than rerunning detection on the full graph.

The `CommunitySummarizer` generates natural-language summaries via LLM (with rule-based fallback), enabling the DRIFT global search path to use community summaries as map-reduce intermediaries.

9 Research Paper Intelligence

9.1 PDF Ingestion Pipeline

The PDF module provides a two-tier parsing architecture:

- **Fast mode**: Uses `pdf-extract` for rapid text extraction (1–5ms per page). Suitable for bulk ingestion.
- **Structured mode**: Heuristic-based academic section detection that identifies title, authors, abstract, sections (Introduction, Methods, Results, etc.), references, equations, and figures via regex patterns on formatting cues.
- **Auto mode**: The `DocumentClassifier` analyzes the raw text to estimate column count, detect academic indicators (abstract, references, affiliations), and check for mathematical symbols, then recommends the optimal strategy.

The `SmartChunker` splits text into semantically coherent chunks with configurable maximum tokens (default 512) and sentence overlap (default 2), respecting section boundaries.

9.2 Multi-Source Academic Search

The `papers` module provides unified search across three academic APIs through the `UnifiedPaperSearch` aggregator:

Table 7: Academic search API integrations.

Source	API	Features
arXiv	Atom XML feed	Categories, PDFs
Semantic Scholar	REST v1	Citations, refs
PubMed	E-utilities	MeSH, PMIDs

Results are deduplicated by title similarity and can be sorted by relevance, citation count, or date. The CLI supports `-since` year filtering and `-download` for automatic PDF retrieval.

9.3 RAG Chat Engine

The chat module implements a complete conversational RAG pipeline:

- **Context Assembly**: The `ContextBuilder` retrieves the top-k relevant chunks from the knowledge graph, formats them with source citations (paper title, section, page number).
- **Conversation History**: The `ConversationHistory` maintains a sliding window of messages with role annotations (User, Assistant, System) and attached source citations.

- **RAG Pipeline**: Formats a system prompt incorporating retrieved context, conversation history, and applicable procedural rules from L4 memory.
- **REPL**: Terminal-based interactive loop with `/clear`, `/quit` commands and colored output.

10 REST API and Dashboard

10.1 API Design

The server module provides a RESTful API built on Axum 0.8 with Tower middleware for CORS and tracing:

Table 8: REST API endpoints.

Verb	Endpoint	Description
GET	`/health`	Health check
GET	`/stats`	Graph statistics
POST	`/papers/ingest`	PDF upload
POST	`/papers/ingest-text`	Text ingestion
POST	`/papers/ingest-url`	URL ingestion
GET	`/papers`	List papers
POST	`/query`	NL query
GET	`/graph`	Graph overview
POST	`/chat`	RAG chat
POST	`/search/papers`	Paper search
POST	`/search/entities`	Entity search

All paths prefixed with `/api/v1`.

All responses follow a uniform `ApiResponse{success, data, error}` envelope. The server state (`AppState`) wraps a shared `NeuroGraph` instance, paper metadata, and conversation histories using `parking_lot::RwLock` for synchronization.

10.2 Embedded Dashboard

Static dashboard assets are embedded in the binary via `rust-embed`, enabling a single-binary deployment with no external file dependencies. The dashboard provides a G6-based graph visualization with node type encoding, importance sizing, and memory tier coloring.

10.3 Model Context Protocol (MCP)

The `neurograph-mcp` crate exposes NeuroGraph as an MCP server, enabling integration with Claude, Cursor, and VS Code Copilot. The MCP server wraps the `neurograph-core` API and provides tool definitions for `add_text`, `query`, `search_papers`, `entity_history`, and `stats`. Transport is supported over both `stdio` (for IDE integration) and Server-Sent Events (SSE) for web-based clients.

11 Embedding Subsystem

NeuroGraph supports four embedding providers behind the `Embedder` trait:

1. **HashEmbedder** (default): Deterministic 128-dim vectors via SeaHash feature hashing. Zero-cost, no API keys, suitable for structural deduplication.
2. **OpenAI Embedder**: `text-embedding-3-small` via the `async-openai` crate.
3. **Ollama Embedder**: Local models (e.g., `nomic-embed-text`) via Ollama's HTTP API.
4. **Cached Embedder**: LRU cache wrapper (default capacity 10,000) around any provider, keyed by BLAKE3 content hash.

The `EmbeddingRouter` selects the appropriate provider based on configuration and falls back gracefully if a provider is unavailable.

12 Command-Line Interface

The CLI (`neurograph-cli`) provides 12 subcommands:

```
neurograph ingest --pdf paper.pdf
neurograph ingest --dir ./papers/ --
    strategy auto
neurograph ingest --url https://
    arxiv.org/pdf/...
neurograph query "Who works at
    Anthropic?" --at 2024
neurograph search "attention" --
    arxiv --s2 --since 2023
neurograph chat --model ollama:
    llama3.2
neurograph dashboard --port 8000 --
    open
neurograph serve --port 8000
neurograph stats
neurograph history "Alice"
neurograph what-changed 2024-01-01
    2025-01-01
neurograph communities
neurograph bench
neurograph info
```

The CLI uses `clap` 4 for argument parsing, `colored` for terminal output, and `indicatif` for progress bars during bulk ingestion.

13 Deployment and Infrastructure

13.1 Build and Distribution

NeuroGraph supports multiple distribution channels:

- **Cargo**: `cargo install neurograph-cli`
- **Docker**: Multi-stage Dockerfile with Rust builder and minimal runtime image
- **Docker Compose**: Full stack with Neo4j, Ollama, and dashboard

13.2 Security and Compliance

The project implements comprehensive security practices: `cargo-deny` for dependency auditing, OSSF Security Insights YAML, a responsible AI transparency document, and a detailed SECURITY.md with vulnerability disclosure procedures.

14 Related Work

NeuroGraph draws inspiration from and extends several systems:

- **Microsoft GraphRAG** [1]: Hierarchical Leiden community detection and global/local search. NeuroGraph extends this with DRIFT adaptive switching and five-strategy hybrid retrieval.
- **Graphiti** [2]: Temporal knowledge graph with episode-based ingestion. NeuroGraph adds bitemporal validity, branch isolation, and RL-guided forgetting.
- **Cognee**: Pipeline-oriented knowledge processing. NeuroGraph adopts the clean `add/query` API pattern.
- **LightRAG**: Lightweight RAG with knowledge graphs. NeuroGraph provides richer retrieval and memory management.
- **Mem0/EverMemOS**: Tiered memory with L1–L4 hierarchy. NeuroGraph implements the tiered model and adds RL-guided evolution.
- **A-MEM**: Zettelkasten self-organizing memory. NeuroGraph implements the keyword evolution insight with Ebbinghaus forgetting integration.
- **MAGMA**: Multi-graph memory architecture. NeuroGraph implements four orthogonal views with intent-aware routing and RRF fusion.

15 Conclusion and Future Work

NeuroGraph demonstrates that a single, cohesive system can integrate temporal knowledge management, advanced hybrid retrieval, biologically-inspired memory evolution, and multi-agent collaboration. By implementing the entire stack in Rust, we achieve both developer ergonomics (via the builder pattern and zero-config defaults) and production-grade performance (async I/O, lock-free concurrency).

Future Directions.

- **HNSW Vector Index**: Replace brute-force similarity search with approximate nearest neighbors for sub-linear scaling.
- **Neo4j Driver**: Full Cypher-backed persistent storage for enterprise deployments.
- **WASM/Python Bindings**: Cross-language SDKs via FFI and WebAssembly compilation.
- **Agentic Framework**: Multi-agent orchestration with role-based branch policies and supervisor approval workflows.
- **Continuous Learning**: Online RL policy updates from user retrieval feedback, closing the forgetting optimization loop.

References

[1] Edge, D. *et al.* From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[2] Zep AI. Graphiti: Building Real-Time, Temporally Aware Knowledge Graphs. *GitHub: getzep/graphiti*, 2024.

[3] Xu, Z. *et al.* A-MEM: Agentic Memory for LLM Agents. *arXiv preprint*, 2025.

[4] Zhang, L. *et al.* EverMemOS: Tiered Memory Operating System for Lifelong AI Agents. *arXiv preprint*, 2026.

[5] Microsoft Research. DRIFT Search: Dynamic Reasoning and Inference with Flexible Traversal. *GraphRAG v3 Documentation*, 2026.

[6] Guo, Z. *et al.* LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint*, 2024.

[7] Cormack, G.V., Clarke, C.L.A., and Buettcher, S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. *Proc. SIGIR*, 2009.

[8] Traag, V.A., Waltman, L., and van Eck, N.J. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports*, 9(1):5233, 2019.

[9] Ebbinghaus, H. Über das Gedächtnis. Duncker & Humblot, Leipzig, 1885.